

Adult Census Income Prediction Using CNN

Project 2 Report

Abstract

This project implements a 1D Convolutional Neural Network (CNN) model to predict whether an individual's annual income exceeds \$50K based on census data from the 1994 Census bureau database. The model processes 14 features including demographic information, education level, occupation, and other relevant attributes. After preprocessing the data and training the CNN model, we achieved a validation accuracy of approximately 84.62%. This report details the methodology, implementation, results, and analysis of the project.

1. Introduction

Income prediction is a classic machine learning problem that has significant applications in economics, social science, and policy making. The goal of this project is to develop a classification model that can accurately predict whether an individual's income exceeds \$50K per year based on various demographic and socioeconomic features.

Problem Statement: Given a dataset containing 14 features for each individual including age, education, occupation, marital status, and other attributes, predict whether the individual's annual income exceeds \$50K. This is a binary classification problem where the target variable is 0 (income $\leq$ \$50K) or 1 (income $>$ \$50K).

2. Dataset Description

The dataset originates from the 1994 Census bureau database, extracted by Ronny Kohavi and Barry Becker. The training set contains 22,792 samples, and the test set contains 9,769 samples. Each sample has 14 features divided into numerical and categorical types.

2.1 Feature Description

Feature	Type	Description
age	Numerical	Working age of the individual
workclass	Categorical	Type of employment (Private, Government, etc.)
fnlwgt	Numerical	Number of people the census represents
education	Categorical	Highest level of education
education.num	Numerical	Number of years of education
marital.status	Categorical	Marital status of the individual

occupation	Categorical	Type of occupation
relationship	Categorical	Family relationship status
race	Categorical	Race of the individual
sex	Categorical	Gender of the individual
capital.gain	Numerical	Capital gains in a year
capital.loss	Numerical	Capital losses in a year
hours.per.week	Numerical	Weekly working hours
native.country	Categorical	Country of origin

3. Data Preprocessing

Data preprocessing is crucial for achieving good model performance. The following steps were applied:

3.1 Missing Value Handling

The dataset contains missing values represented by '?' characters. For categorical features, missing values were imputed using the mode (most frequent value). For numerical features, missing values were imputed using the median value. This approach preserves the distribution characteristics of the data while ensuring no information is lost.

3.2 Categorical Feature Encoding

Categorical features were encoded using Label Encoding. Each unique category was assigned a numerical value. The encoder was fitted on the combined training and test data to ensure consistency and handle any categories that might only appear in one set. Eight categorical features were encoded: workclass, education, marital.status, occupation, relationship, race, sex, and native.country.

3.3 Feature Scaling

All features were standardized using StandardScaler to have zero mean and unit variance. This normalization is essential for neural networks as it helps with gradient descent optimization and ensures that features with larger scales don't dominate the learning process.

4. CNN Model Architecture

A 1D Convolutional Neural Network (CNN) was chosen for this classification task. While CNNs are traditionally used for image data, 1D CNNs can effectively capture local patterns in tabular data by treating features as a sequence. The model architecture is designed to extract meaningful representations from the input features through multiple convolutional layers.

4.1 Architecture Details

The CNN model consists of the following components:

Input Layer: Receives 14 preprocessed features, reshaped to (batch_size, 1, 14) for 1D convolution.

Convolutional Block 1: - 1D Convolution: 64 filters, kernel size 3, padding 1 - Batch Normalization - ReLU Activation - Dropout (rate=0.3)

Convolutional Block 2: - 1D Convolution: 128 filters, kernel size 3, padding 1 - Batch Normalization - ReLU Activation - Dropout (rate=0.3)

Convolutional Block 3: - 1D Convolution: 64 filters, kernel size 3, padding 1 - Batch Normalization - ReLU Activation - Dropout (rate=0.3)

Global Average Pooling: Reduces spatial dimensions to a single value per channel.

Fully Connected Layers: - Dense layer: 64 → 32 neurons with ReLU activation - Dropout (rate=0.3) - Output layer: 32 → 1 neuron with Sigmoid activation

The model contains a total of 52,225 trainable parameters.

4.2 Design Rationale

Why CNN for Tabular Data? While traditional machine learning models like Random Forest or XGBoost are commonly used for tabular data, CNNs offer several advantages:

1. **Local Pattern Detection:** The convolution operation can detect local feature interactions that might be missed by fully connected networks.
2. **Parameter Sharing:** Weight sharing in convolutions provides regularization.
3. **Hierarchical Feature Learning:** Multiple layers learn increasingly abstract features.

Regularization Techniques:

- **Batch Normalization:** Stabilizes training and allows higher learning rates
- **Dropout (30%):** Prevents overfitting by randomly dropping neurons during training
- **Weight Decay (L2 regularization):** Added to the optimizer to prevent overfitting

5. Training Configuration

The model was trained with the following configuration:

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Weight Decay	1e-4
Batch Size	64
Loss Function	Binary Cross-Entropy
Maximum Epochs	100
Early Stopping Patience	15 epochs
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=5)
Train/Val Split	80% / 20% (stratified)
Random Seed	42

Early Stopping: Training was stopped when the validation loss did not improve for 15 consecutive epochs. The model with the best validation loss was saved and used for final predictions. In our experiment, training stopped at epoch 73 out of 100.

Learning Rate Scheduling: The learning rate was reduced by a factor of 0.5 when the validation loss plateaued for 5 epochs, helping the model converge to a better optimum.

6. Experimental Results

6.1 Training Progress

The model was trained for 73 epochs before early stopping was triggered. The training and validation metrics showed consistent improvement with the following final results:

- Final Training Loss: 0.3267 - Final Training Accuracy: 84.77% - Final Validation Loss: 0.3219 - Final Validation Accuracy: 84.62%

6.2 Evaluation Metrics

Class	Precision	Recall	F1-Score	Support
≤50K	0.87	0.93	0.90	3461
>50K	0.73	0.57	0.64	1098
Weighted Avg	0.84	0.85	0.84	4559

6.3 Confusion Matrix Analysis

The confusion matrix for the validation set:

- True Negatives (correctly predicted ≤50K): 3,229 - False Positives (≤50K predicted as >50K): 232 - False Negatives (>50K predicted as ≤50K): 469 - True Positives (correctly

predicted >50K): 629

Analysis: The model shows better performance on the majority class ($\leq 50K$) with 93% recall compared to the minority class ($> 50K$) with 57% recall. This is a common challenge in imbalanced classification problems where the $\leq 50K$ class has approximately 3x more samples than the $> 50K$ class.

6.4 Test Set Predictions

The trained model generated predictions for all 9,769 test samples:

- Predicted $\leq 50K$: 7,868 samples (80.5%) - Predicted $> 50K$: 1,901 samples (19.5%)

This distribution is consistent with the expected class distribution in the Adult Census dataset.

7. Discussion

7.1 Model Performance Analysis

The CNN model achieved 84.62% validation accuracy, which is competitive for this dataset. The performance metrics reveal several insights:

Strengths:

- High overall accuracy (84.62%) - Good precision and recall for the majority class - Stable training with gradual improvement - Effective regularization preventing significant overfitting

Challenges:

- Lower recall for the minority class ($> 50K$) indicating difficulty in identifying high-income individuals - Class imbalance affects model's ability to generalize for the minority class

7.2 Comparison with Traditional Methods

While traditional machine learning methods like Random Forest, Gradient Boosting, or Support Vector Machines often achieve similar or slightly higher accuracy on this dataset (typically 85-87%), the CNN approach offers:

1. Automatic feature learning without extensive feature engineering 2. Ability to capture non-linear feature interactions 3. Scalability to larger datasets 4. Flexibility to incorporate additional data modalities

8. Limitations and Future Work

8.1 Current Limitations

1. **Class Imbalance:** The model struggles with the minority class due to the imbalanced dataset (approximately 75% $\leq 50K$ vs 25% $> 50K$).

2. **Feature Engineering:** The current approach uses basic preprocessing without advanced feature engineering techniques.

3. **Architecture Optimization:** The CNN architecture was not extensively tuned through hyperparameter optimization.

4. **Limited Interpretability:** Deep learning models are less interpretable compared to traditional methods like decision trees.

8.2 Future Improvements

1. Address Class Imbalance:

- Implement class weights in the loss function - Use oversampling (SMOTE) or undersampling techniques - Apply focal loss to focus on hard examples

2. Model Enhancements:

- Experiment with deeper architectures or residual connections - Try TabNet or other architectures designed for tabular data - Implement attention mechanisms for feature importance

3. Ensemble Methods:

- Combine CNN with traditional models (Random Forest, XGBoost) - Use model stacking or blending for improved predictions

4. Advanced Preprocessing:

- Feature interaction creation - Target encoding for categorical variables - Polynomial feature generation

9. Conclusion

This project successfully implemented a 1D Convolutional Neural Network for the Adult Census Income prediction task. The model achieved 84.62% validation accuracy, demonstrating the feasibility of using CNN architectures for tabular classification problems.

The preprocessing pipeline effectively handled missing values, encoded categorical features, and normalized the data for optimal neural network training. The model architecture, combined with regularization techniques (dropout, batch normalization, early stopping), provided stable training without significant overfitting.

While there is room for improvement, particularly in handling class imbalance and optimizing the architecture, this project demonstrates that CNNs can be effectively applied to structured tabular data classification tasks beyond their traditional image processing applications.

References

1. Kohavi, R., & Becker, B. (1996). UCI Machine Learning Repository: Adult Data Set.
2. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
3. Srivastava, N., et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(1), 1929-1958.
4. Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training. *ICML 2015*.